

运行时差分测试

主要完成了一些针对运行时 `solc` (evm) 和 `solang` (Polkadot WASM) 的差分测试基础设施验证。

当前解决方案

1. 使用 `solc` 将合约编译为 EVM 字节码。
2. 使用 `solang --target Polkadot` 编译为 WASM 字节码和 `.contract` 元数据。
3. 使用 EVM 运行时运行 `solc` 产物。
4. 使用 Polkadot 运行时部署 `solang` 编译产物。
5. 解码两个运行时的输出，从而获取一个可观测的共性数据。

主要对比内容包括：

- 交易是否成功。
- 解码后的返回值是否一致。
- 交易失败的解码返回情况（如果计划失败）。
- 事件记录，并解码事件。
- 存储信息，包括账户信息和全局信息等。
- 数值差异表现，例如交易的数值。

最小范例

```
// SPDX-License-Identifier: UNLICENSED
// Counter.sol
pragma solidity ^0.8.20;

contract Counter {
    uint256 private total;

    event Incremented(address indexed caller, uint256 delta, uint256 value);

    constructor(uint256 initial) payable {
        total = initial;
    }

    function get() external view returns (uint256) {
        return total;
    }

    function increment(uint256 delta) external payable {
        require(delta > 0, "zero increment");
    }
}
```

```
        total += delta;
        emit Incremented(msg.sender, delta, msg.value);
    }
}
```

场景：

```
deploy Counter(5) from alice with value 10
query get() → 5
tx increment(3) from bob with value 7
query get() → 8
tx increment(0) → revert
```

如何操作

编译 EVM：

```
solc --bin --abi --overwrite -o evm-out Counter.sol
```

编译 Polkadot：

```
solang compile --target polkadot --output polkadot-out Counter.sol
```

运行 Polkadot：

```
uv run solidity-diff-fuzz transaction-smoke Counter.sol --work-dir
/tmp/smoke
```

随后可以查看：

```
substrate-contracts-node --dev --tmp --rpc-port 9944 --rpc-cors all
```

产物

```
case: Counter.sol
status: equivalent-observations
steps:
  - step: 1
    kind: query
    message: get
    evm: {status: success, returns: [5]}
    polkadot: {status: success, returns: [5]}
  - step: 2
    kind: tx
    message: increment
```

```
    evm: {status: success, events: [Incremented]}
    polkadot: {status: success, events: [Incremented]}
- step: 3
  kind: query
  message: get
  evm: {status: success, returns: [8]}
  polkadot: {status: success, returns: [8]}
- step: 4
  kind: tx
  message: increment
  evm: {status: revert, error: Error(string)}
  polkadot: {status: revert, error: Error(string)}
```

后续计划

目前只搭建了部分架构，如果需要考虑部分自动/半自动的设计，可能需要进一步构建合约的自动化部署、上传等内容。

同时目前没有针对 EVM 交叉验证，有待后续处理。